

Monkey-Head-Project Repository Reorganization Report

Executive summary

The repository's core problem is not just sprawl. It is **canon drift plus mixed strata**. The public website still presents Huey as a broad embodied platform with dual displays, constitutional governance, PyGPT-net, Ollama, custom kernels, and phased embodiment, while the current `README.md` frames the active target as a much narrower **Huey Brain V1** running on a Lenovo Legion Go with a deterministic MP3 → transcription → cognition → structured-log loop. At the same time, the repo still exposes a machine-facing master plan file at `master-plan-v30.2.json`, even though the README claims the canonical machine-facing spec is `master-plan-v32.0.json`. That means public narrative, repo front door, and machine-facing canon are not synchronized. 1 2 3

The second problem is **layer collision inside the tree**. The repo mixes active Python runtime, legacy runtime, constitutional texts, media assets, release payloads, installer/boot binaries, vendored overlays, submodules, and security documentation in ways that blur what is live, what is reference, and what is archival. The most overloaded example is `src/huey/memory/`, which currently acts as a code location, a canon shelf, an asset vault, and an archive bucket at once. The repo also carries substantial binary and release payload surface, including `initrd.gz`, kernel images, `.deb` packages, `.udeb` pools, videos, and zip archives; GitHub recommends keeping repos small, warns on files above 50 MiB, blocks files above 100 MiB, and explicitly recommends Git LFS or Releases for large binaries. 4 5 6

The best reorganization is therefore **layer-first, not technology-first**. I recommend a top-level structure that separates: `canon/` for authoritative present-tense truth, `src/` and `infra/` for implementation, `experiments/` for non-canonical lab work, `history/` for frozen lineage and payload archives, `docs/` for explanatory and operational documentation, `assets/` for static media, and `third_party/` for submodules and overlays. This keeps lineage intact because the migration should use `git mv` **rather than copy/delete**, baseline tags before restructuring, pointer READMEs where old paths vanish, and release/tag mapping for external references. Git's own documentation confirms that `git mv` updates the index and, for submodules, also updates `.gitmodules` path metadata. 7 8 9

I recommend doing this in **two phases**. Phase one should separate canon, history, docs, assets, third-party, and large artifacts **without** changing package names or runtime imports. Phase two should tackle the harder namespace cleanup (`huey` versus `hueyos`) only after the tree is structurally clean and the test baseline is stabilized. That sequencing matters because the current package/test surface already relies on import shims and mixed namespaces, and the repo's own recent security remediation summary notes that `pytest -q` is not currently green in the tested environment due to collection/import issues. 10 11 12

13

What the repository is signaling now

The live materials currently point in different directions. The website says the public “start here” source of truth is GitHub and describes a broad, embodied HueyOS stack with phased embodiment and constitutional autonomy. The README, by contrast, says the active direction is Huey Brain V1 on a Legion Go, with Body, HIMS runtime, PyGPT-net, and distributed compute explicitly deferred. The machine-facing plan visible in the repo is still `master-plan-v30.2.json`, whose content is older than the README’s claimed `v32.0` canon. That is the strongest evidence that any reorganization must start by separating **historical**, **canonical**, and **implementation** materials before doing anything else. 1 2 3

The active implementation surface is real, but it is mixed with legacy and doctrinal layers. The repo has a Python package defined in `pyproject.toml` as `hueyos` with Python `>=3.13, <3.15`, a FastAPI-facing runtime surface, a CLI entrypoint, large optional dependency groups, and active CI/test/security workflows. But the tests also reveal import-path accommodation between `huey`, `huey.memory.PY`, and `src`, which is a sign that the runtime boundary is not yet cleanly isolated. The security remediation summary further states that the broader suite currently fails collection in the tested environment, which reinforces the need to separate structural cleanup from namespace refactoring. 12 14 10 11 13

The tree also includes prior restructuring work products that are useful evidence. `docs/unsorted/repository-restructure-recommendation.md` already proposes separating source, generated artifacts, release payloads, and ownership boundaries. `docs/unsorted/repository-restructure-inventory.md` already lists concrete moves such as pushing boot/install artifacts under `platform/` and releases under `archives/`, consolidating PyGPT integrations, and resolving `src/huey` versus `src/hueyos`. Those documents are directionally correct, but they remain explanatory documents rather than an enforced canonical structure. 7 8

Verified audit of high-signal files and path families

The available live connectors did not expose a lossless recursive tree API with exact size and last-modified metadata for every tracked file. The repository itself does contain a generated `git ls-files` inventory and a recent artifact-surface report, which together make the overall content families clear, but not every long-tail per-file date could be confirmed interactively. Where exact values were not retrievable from live metadata, I mark them as **unspecified** rather than guessing. 15 4

Path or family	Type	Size or count	Last-modified evidence	Inferred purpose	Proposed layer
<code>README.md</code>	Markdown	unspecified	active on <code>main</code> ; version text claims <code>v32.0</code>	human-facing front door and current-state narrative	<code>canon/</code> pointer at root + source in <code>canon/</code>
<code>master-plan-v30.2.json</code>	JSON	unspecified	exact file present on <code>main</code>	machine-facing plan/spec, but version-drifted	<code>canon/machine/</code>

Path or family	Type	Size or count	Last-modified evidence	Inferred purpose	Proposed layer
<code>src/huey/memory/TXT/*.txt</code>	text chapters	unspecified	referenced by README as canonical law/ front matter; commit history added them in April 2026	constitutional canon and glossary/book matter	<code>canon/law/</code>
<code>src/huey/memory/PY/api.py</code>	Python	large active file	active implementation file	FastAPI control surface; broad runtime scope	<code>src/</code> or <code>implementation/</code>
<code>src/huey/memory/PY/run.py</code>	Python	large active file	active implementation file	runtime launcher / PyGPT integration path	<code>src/</code> or <code>implementation/</code>
<code>tests/test_cli.py</code>	Python test	unspecified	active test file	CLI behavior tests	<code>tests/</code>
<code>tests/test_api_routes.py</code>	Python test	unspecified	active test file	API behavior/ security tests, currently shim-heavy	<code>tests/</code>
<code>.github/workflows/ci.yml</code>	YAML	unspecified	active workflow	lint + tests on <code>main</code> and <code>develop</code>	<code>.github/</code>
<code>.github/workflows/codeql.yml</code>	YAML	unspecified	active workflow	code scanning on <code>main</code>	<code>.github/</code>
<code>.github/workflows/security-bandit.yml</code>	YAML	unspecified	active workflow	Bandit scan	<code>.github/</code>
<code>.github/workflows/security-dependency-scan.yml</code>	YAML	unspecified	active workflow	<code>pip-audit</code> scan	<code>.github/</code>
<code>.github/workflows/security-artifact-container-scan.yml</code>	YAML	unspecified	active workflow	large-file / Trivy artifact scan	<code>.github/</code>
<code>.github/CODEOWNERS</code>	text	unspecified	active file	review ownership, but currently stale path globs	<code>.github/</code>

Path or family	Type	Size or count	Last-modified evidence	Inferred purpose	Proposed layer
<code>.gitattributes</code>	text	unspecified	active file	line ending policy, binary treatment, archive/export flags; contains stale vendored path rule	root policy file
<code>.gitmodules</code>	text	unspecified	active file	submodule declaration for <code>integrations/pygpt/py-gpt</code>	root policy file
<code>integrations/pygpt/py-gpt</code>	submodule	unspecified	active submodule path	upstream third-party dependency	<code>third_party/pygpt/upstream/</code>
<code>integrations/pygpt/pygpt-mhp</code>	vendored overlay	unspecified	active path	project-specific overlay/ customization around PyGPT	<code>third_party/pygpt/overlay/</code>
<code>docs/unsorted/index.md</code>	Markdown	unspecified	active file	documentation landing page, but still under <code>unsorted</code>	<code>docs/</code>
<code>docs/unsorted/governance.md</code>	Markdown	unspecified	active file	governance/ resilience explanation, but cross-references older paths	<code>docs/architecture/</code>
<code>docs/security/*</code>	Markdown set	growing	several files added/updated recently	security policy, status, incident notes	<code>docs/security/</code>
<code>apps/huey-gui/*</code>	Python/UI assets	unspecified	active path	GUI surface; unclear if current production or prototype	<code>experiments/gui/</code> unless explicitly current
<code>config/main.config</code>	JSON-style config	tiny	active file	logging config	<code>implementation/config/</code> or keep <code>config/</code>

Path or family	Type	Size or count	Last-modified evidence	Inferred purpose	Proposed layer
<code>archives/releases/**</code>	binaries/checksums	multiple files	historical payloads	published release artifacts and checksums	<code>history/releases/</code> + GitHub Releases
<code>platform/installers/linux/gtk/initrd.gz</code>	generated binary	63.4 MB	size from artifact report	installer boot image	GitHub Release asset or LFS; not normal Git
<code>platform/installers/linux/initrd.gz</code>	generated binary	23.2 MB	size from artifact report	installer boot image	GitHub Release asset or LFS
<code>platform/installers/linux/gtk/vmlinuz</code>	generated binary	11.5 MB	size from artifact report	GTK installer kernel	GitHub Release asset or LFS
<code>platform/installers/linux/vmlinuz</code>	generated binary	11.5 MB	size from artifact report	installer kernel	GitHub Release asset or LFS
<code>platform/boot/legacy/live/vmlinuz-6.18.5-hueyos</code>	generated binary	12.2 MB	size from artifact report	live kernel payload	Release asset, with manifest in Git
<code>platform/boot/legacy/live/vmlinuz</code>	generated binary	12.2 MB	size from artifact report	likely duplicate default-path kernel	review for dedupe, then release asset
<code>archives/releases/hueyos-6.18.5.../*.deb</code>	<code>.deb</code> packages	17.8 MB, 8.9 MB examples	size from artifact report	archived kernel/header payloads	<code>history/releases/</code> + GitHub Releases
<code>platform/packaging/pool-udeb/**</code>	<code>.udeb</code> pool	232 files, ~55.1 MB	size/count from artifact report	offline installer dependency pool	keep only if offline mirror is a first-class product; otherwise externalize
<code>platform/boot/grub/grub/x86_64-efi/**</code>	GRUB modules	279 files, ~2.9 MB	size/count from artifact report	prebuilt EFI boot module set	keep only if deterministic regeneration is not yet available
<code>src/huey/memory/ZIP/dlrrp-ca-v84.3.zip</code>	ZIP archive	38.1 MB	size from artifact report	reference archive / bundled asset	<code>history/assets/zip/</code> or release asset

Path or family	Type	Size or count	Last-modified evidence	Inferred purpose	Proposed layer
<code>src/huey/memory/MP4/TEST.mp4</code>	video	11.1 MB	size from artifact report	media asset / test content	<code>assets/</code> or <code>history/assets/video/</code>
<code>assets/images/HueyOS-background.png</code>	image	unspecified	present in generated inventory	static image asset	<code>assets/images/</code>

Evidence for the artifact-heavy side is especially strong. The artifact report identifies `initrd.gz`, `vmlinux`, `.deb`, `.udeb`, ZIP, MP4, EFI, and GRUB module families as committed payload surface, and recommends moving many of them to release channels or other external storage. GitHub's own documentation aligns with that recommendation: regular repositories warn above 50 MiB, block above 100 MiB, recommend staying ideally under 1 GB and strongly under 5 GB at the old guidance, and now recommend a 10 GB on-disk ceiling with generated files stored outside Git and binary payloads tracked with LFS where appropriate. [4](#) [16](#)

Classification rules by layer

Because the repo contains both explicitly verified files and very large path families, the cleanest classification method is **path-rule classification**: every file in a subtree inherits the layer unless explicitly overridden.

Layer	What belongs there now	Why
Canonical	<code>README.md</code> , source content, <code>master-plan-v30.2.json</code> , constitutional/front-matter text chapters currently under <code>src/huey/memory/TXT/</code>	present-tense truth, law, machine-facing spec, public front door
Implementation	runtime Python packages, active CLI/API code, active configs, active infra, active tests	runnable software and operational surfaces
Experimental	GUI prototypes, lab scripts, fixtures, exploratory model/training notes, optional prototype apps	real lab work, but non-canonical and not guaranteed stable
Historical	archived releases, frozen old plans, deprecated prompts, legacy GUI/runtime materials, old distribution payloads	lineage must be preserved, but these should not compete with active code
Docs	explanatory runbooks, architecture notes, contributor guidance, security documentation, release notes	important but not themselves canon or runtime
Assets	images, icons, videos, public-facing media	static non-code content

Layer	What belongs there now	Why
CI	<code>.github/workflows/*</code> , CI support scripts	automation and repository policy
Tests	<code>tests/*</code> , test-specific fixtures	verification only
Third-party	<code>integrations/pygpt/py-gpt</code> submodule and project-specific overlay	upstream or externally derived code with distinct maintenance cadence

That classification also exposes two existing policy drift issues. First, `.gitattributes` marks `memory/**` as generated but the repo's high-value canonical and asset content is actually inside `src/huey/memory/**`, so the semantics do not match the live tree. Second, `.gitattributes` marks `repo/py-gpt/**` as vendored, but the live submodule path is `integrations/pygpt/py-gpt`, so Linguist/vendor policy is stale. `.github/CODEOWNERS` has a similar drift problem: it assigns `/src/huey/api/` rather than the live file-oriented runtime surface now exposed in the repo. [5](#) [17](#) [18](#)

Proposed layered target structure

The least-destructive structure is one that preserves the familiar root operational surfaces (`src/`, `tests/`, `.github/`, `infra/`) while moving canon, history, experiments, and third-party into unmistakable top-level layers.

```

.
├── .github/           # workflows, CODEOWNERS, templates, rulesets docs
├── assets/            # images, icons, public/static media
├── canon/             # authoritative present-tense truth
│   ├── machine/       # master plans, machine-facing specs
│   ├── law/           # constitution, glossary, chapters, doctrine
│   └── public/        # README source fragments, public-facing summaries
├── docs/              # architecture, runbooks, contributor/security docs
│   ├── architecture/
│   ├── runbooks/
│   ├── contributor/
│   ├── security/
│   └── reference/
├── experiments/       # prototypes, fixtures, exploratory work
│   ├── gui/
│   ├── fixtures/
│   ├── prototype-code/
│   └── notebooks/
├── history/           # preserved lineage, frozen snapshots, old payloads
│   ├── releases/
│   ├── docs/
│   └── prompts/

```

```

|   |— assets/
|   |— platform/
|— infra/           # active deployment, containers, secrets templates
|— src/             # active implementation only
|— tests/           # active tests only
|— third_party/     # submodules, overlays, vendored integrations
|— config/          # active runtime config templates

```

This model makes one architectural promise: **nothing under `canon/` is experimental, nothing under `history/` is active by default, and nothing under `src/` is doctrinal storage**. That single promise would remove most of the current ambiguity. It also aligns naturally with the current public needs: the website can point to `canon/` for truth, the README can be generated or checked against `canon/public/`, and releases can point to tagged `canon/` plus implementation snapshots. ¹⁹ ²

Layer relationships

```

flowchart TD
    CANON[canon<br/>authoritative truth]
    DOCS[docs<br/>explanatory + operational]
    IMPL[src + infra + config<br/>active implementation]
    TESTS[tests<br/>verification]
    EXP[experiments<br/>non-canonical lab work]
    HIST[history<br/>frozen lineage]
    TP[third_party<br/>submodules + overlays]
    ASSETS[assets<br/>static media]
    SITE[dlrp.ca + GitHub Releases]

    CANON --> DOCS
    CANON --> IMPL
    IMPL --> TESTS
    EXP -. proven ideas promoted .-> IMPL
    EXP -. accepted doctrine promoted .-> CANON
    IMPL -. deprecated or superseded .-> HIST
    CANON -. superseded versions .-> HIST
    TP --> IMPL
    ASSETS --> DOCS
    ASSETS --> SITE
    CANON --> SITE
    HIST --> SITE

```

Naming conventions

Use naming that encodes layer and stability, not just topic.

Rule	Recommendation
Directories	lowercase kebab-case for non-Python dirs
Python packages	keep Python package naming stable during phase one; decide <code>huey</code> versus <code>hueyos</code> only in phase two
Canon files	numeric prefixes only where order matters materially, such as constitutional chapters
Historical snapshots	date or version prefix, for example <code>history/releases/hueyos-6.18.5/</code> or <code>history/canon/2026-04-v30.2/</code>
Experiments	prefix directories by theme, not outcome, for example <code>experiments/fixtures/</code> , <code>experiments/gui/</code> , <code>experiments/model-bench/</code>
Assets	group by media type or consumer, for example <code>assets/images/</code> , <code>assets/icons/</code> , <code>assets/site/</code>
Third-party	<code>third_party/<vendor-or-project>/<kind>/</code> such as <code>third_party/pygpt/upstream</code> and <code>third_party/pygpt/overlay</code>

Migration plan and lineage preservation

The migration should preserve history through **small-batch** `git mv` **commits**, not a single mega-rename. Git documents that `git mv` updates the index and stages the rename; it also updates `.gitmodules` path information when moving a submodule. GitHub Releases are the right place for deployable binaries attached to a tagged point in history. ²⁰

Recommended branch and tag strategy

Branch or tag	Purpose
<code>main</code>	protected stable branch
<code>reorg/layers-2026-05</code>	integration branch for the structure split
<code>pre-layer-split-2026-05-03</code>	immutable baseline tag before moves
<code>canon-sync</code>	optional short-lived branch if canon files are cleaned before code moves
<code>archive/pre-namespace-cleanup</code>	optional tag before <code>huey</code> / <code>hueyos</code> namespace work

Protect `main` with required status checks, at least one non-self review where possible, stale-review dismissal, and required CODEOWNERS approval on canonical, implementation, and release-critical paths. GitHub explicitly supports required status checks, required reviews, CODEOWNERS review requirements, linear history, and merge queue/protected-branch settings for this. ²¹

Step-by-step migration sequence

```

flowchart LR
    A[Tag baseline<br/>pre-layer-split] --> B[Add repo-map + layer READMEs]
    B --> C[Move canon files]
    C --> D[Move docs out of unsorted]
    D --> E[Move history/releases + heavy binaries]
    E --> F[Normalize third_party PyGPT paths]
    F --> G[Update CI path filters + CODEOWNERS]
    G --> H[Run tests + manual smoke checks]
    H --> I[Merge reorg branch]
    I --> J[Optional phase two:<br/>namespace cleanup]
  
```

High-impact path mapping

The table below is the path-preserving core of the migration. For old-path continuity, prefer **README pointer files** or minimal stub docs over symlinks for documentation and media, because pointer files are more predictable in GitHub's web UI and across Windows clones.

Old path	New path	Preserve-lineage note
<code>README.md</code>	keep at root, but source governed from <code>canon/public/README.md</code>	root README stays as front door; canonical source tracked separately
<code>master-plan-v30.2.json</code>	<code>canon/machine/master-plan-v30.2.json</code>	leave a root pointer file or symlink-like README note for one release
<code>src/huey/memory/TXT/00 - TOC_&_Glossary.txt</code>	<code>canon/law/00-toc-and-glossary.txt</code>	exact move with <code>git mv</code> ; update README references
<code>src/huey/memory/TXT/03 - Huey_Constitution.txt</code>	<code>canon/law/03-huey-constitution.txt</code>	exact move with <code>git mv</code> ; protect with CODEOWNERS
other <code>src/huey/memory/TXT/*.txt</code> canon chapters	<code>canon/law/chapters/</code>	preserve numbering where it carries meaning
<code>docs/unsorted/index.md</code>	<code>docs/reference/index.md</code> or <code>docs/index.md</code>	replace <code>unsorted</code> with topical folders
<code>docs/unsorted/governance.md</code>	<code>docs/architecture/governance-runtime.md</code>	keep explanatory docs separate from law
<code>archives/releases/**</code>	<code>history/releases/**</code>	preserve path family; attach deployable binaries to GitHub Releases

Old path	New path	Preserve-lineage note
<code>platform/installers/**</code> generated binaries	release assets + <code>history/platform/installers/manifest/</code>	keep recipes in Git, not heavy outputs
<code>platform/boot/**</code> generated images/modules	<code>history/platform/boot/</code> plus text recipes under active <code>platform/recipes/</code>	split source recipe from generated output
<code>integrations/pygpt/py-gpt</code>	<code>third_party/pygpt/upstream</code>	<code>git mv</code> submodule path; updates <code>.gitmodules</code>
<code>integrations/pygpt/pygpt-mhp</code>	<code>third_party/pygpt/overlay</code>	make overlay explicitly distinct from upstream
<code>src/huey/memory/ZIP/**</code>	<code>history/assets/zip/**</code>	preserve lineage, remove from runtime tree
<code>src/huey/memory/MP4/**</code>	<code>assets/video/**</code> or <code>history/assets/video/**</code>	choose based on whether it is active/public or archival
<code>apps/huey-gui/**</code>	<code>experiments/gui/**</code> unless explicitly production	only keep under implementation if truly current
<code>config/main.config</code>	keep <code>config/main.config</code> or move to <code>implementation/config/main.config</code>	low-risk; can remain if referenced by tooling

Example command sequence

```
git checkout -b reorg/layers-2026-05
git tag pre-layer-split-2026-05-03

mkdir -p canon/machine canon/law/chapters
mkdir -p docs/{architecture,runbooks,contributor,security,reference}
mkdir -p experiments/{gui,fixtures,prototype-code}
mkdir -p history/{releases,docs,prompts,assets,platform}
mkdir -p third_party/pygpt
mkdir -p assets/{images,icons,video}

git mv master-plan-v30.2.json canon/machine/master-plan-v30.2.json
git mv "src/huey/memory/TXT/00 - TOC_&_Glossary.txt" canon/law/00-toc-and-glossary.txt
git mv "src/huey/memory/TXT/03 - Huey_Constitution.txt" canon/law/03-huey-constitution.txt
```

```

git mv docs/unsorted/index.md docs/reference/index.md
git mv docs/unsorted/governance.md docs/architecture/governance-runtime.md
git mv docs/unsorted/repository-restructure-inventory.md docs/contributor/
repository-restructure-inventory.md
git mv docs/unsorted/repository-restructure-recommendation.md docs/contributor/
repository-restructure-recommendation.md

git mv archives/releases history/releases
git mv integrations/pygpt/py-gpt third_party/pygpt/upstream
git mv integrations/pygpt/pygpt-mhp third_party/pygpt/overlay

git mv src/huey/memory/ZIP history/assets/zip
git mv src/huey/memory/MP4 assets/video
git mv apps/huey-gui experiments/gui

git commit -m
"move(layer): split canon docs history experiments and third-party paths"

```

Example migration helper script

```

#!/usr/bin/env bash
set -euo pipefail

# Usage: bash scripts/migrate_layers.sh
# Assumes you are on a dedicated reorg branch.

mkdir -p canon/machine canon/law/chapters
mkdir -p docs/{architecture,runbooks,contributor,security,reference}
mkdir -p experiments/{gui,fixtures,prototype-code}
mkdir -p history/{releases,docs,prompts,assets,platform}
mkdir -p third_party/pygpt
mkdir -p assets/{images,icons,video}

move_if_exists() {
    local src="$1"
    local dst="$2"
    if git ls-files --error-unmatch "$src" >/dev/null 2>&1; then
        mkdir -p "$(dirname "$dst")"
        git mv "$src" "$dst"
    fi
}

move_if_exists "master-plan-v30.2.json" "canon/machine/master-plan-v30.2.json"
move_if_exists "docs/unsorted/index.md" "docs/reference/index.md"
move_if_exists "docs/unsorted/governance.md" "docs/architecture/governance-
runtime.md"

```

```

move_if_exists "docs/unsorted/repository-restructure-inventory.md" "docs/
contributor/repository-restructure-inventory.md"
move_if_exists "docs/unsorted/repository-restructure-recommendation.md" "docs/
contributor/repository-restructure-recommendation.md"
move_if_exists "src/huey/memory/ZIP" "history/assets/zip"
move_if_exists "src/huey/memory/MP4" "assets/video"
move_if_exists "apps/huey-gui" "experiments/gui"

# Submodule path move: git mv updates .gitmodules metadata.
if [ -d "integrations/pygpt/py-gpt" ]; then
    mkdir -p third_party/pygpt
    git mv integrations/pygpt/py-gpt third_party/pygpt/upstream
fi
if [ -d "integrations/pygpt/pygpt-mhp" ]; then
    mkdir -p third_party/pygpt
    git mv integrations/pygpt/pygpt-mhp third_party/pygpt/overlay
fi

echo "Layer migration moves staged. Review with: git status --short"

```

Recommended commit sequence

Commit	Purpose
chore(repo): tag pre-layer-split baseline	freeze rollback point
docs(repo): add repo-map and layer policy	explain structure before moving files
move(canon): relocate master plan and law texts	establish source of truth
move(docs): split unsorted docs into topical folders	make docs navigable
move(history): relocate releases and heavy assets	remove archive pressure from active tree
move(third-party): normalize pygpt upstream and overlay paths	isolate external code
refactor(ci): update workflows codeowners and path filters	keep automation aligned
refactor(runtime): remove compatibility shims	phase two only, after tests stabilize

Artifact, CI/CD, and release handling

The binary-artifact side of this repository should be handled with a much stricter rule: **recipes and manifests in Git; heavy generated outputs in Releases or LFS; submodules for live third-party code; overlays separate from upstream**. That is directly supported by GitHub's guidance on large files, Releases, and LFS object handling in archives. ²²

Large files, binaries, and generated artifacts

Use the following policy:

Artifact class	Recommended home	Why
Installer kernels, <code>initrd.gz</code> , EFI images, ISO payloads	GitHub Releases attached to tags	deployable binaries; large and generated
Frequently-needed binary assets that must stay cloneable	Git LFS	GitHub supports this pattern up to plan-based per-file limits
Old release payloads kept only for lineage	<code>history/releases/</code> or Release assets plus manifest	preserve lineage without cluttering active tree
Deterministically regenerable boot/module sets	build recipe in Git, output in Releases	source-controlled provenance, smaller repo
Upstream PyGPT dependency	submodule	upstream history stays distinct
Project-specific modifications to upstream	overlay directory, not mixed into upstream	clean diff against third-party base
Checksums, manifests, SBOMs	keep in Git	lightweight provenance metadata

The repository's own `ARTIFACT_SURFACE.md` already recommends a checksum-manifest pattern and explicitly identifies several heavy files as better suited to release artifacts. GitHub likewise recommends Releases for distributing large binaries and LFS for files beyond normal Git comfort. If LFS is used, note that GitHub archives exclude LFS objects by default unless repository settings opt in, and those downloads count toward LFS bandwidth. ⁴ ²²

Recommended CI/CD changes

The repo already has useful building blocks: a general CI workflow, CodeQL, Bandit, dependency scanning, and artifact/container-adjacent scanning. The missing step is **path-aware specialization** and **layer-aware required checks**. ²³ ²⁴ ²⁵ ²⁶ ²⁷

I recommend this workflow layout:

Workflow	Trigger paths	Enforced on PRs?	Purpose
<code>ci-runtime.yml</code>	<code>src/**</code> , <code>tests/**</code> , <code>pyproject.toml</code> , <code>requirements.txt</code> , <code>config/**</code>	yes	lint, unit tests, import smoke tests
<code>ci-canon.yml</code>	<code>canon/**</code> , <code>README.md</code> source, <code>docs/architecture/**</code>	yes	canon consistency checks, broken-reference checks
<code>ci-docs.yml</code>	<code>docs/**</code> , <code>README.md</code> , <code>assets/**</code>	yes	markdown lint, link checks
<code>ci-security.yml</code>	<code>src/**</code> , <code>infra/**</code> , <code>.github/**</code> , dependency manifests	yes	Bandit, pip-audit, secret scan, CodeQL
<code>ci-artifact-policy.yml</code>	<code>platform/**</code> , <code>history/releases/**</code> , <code>assets/video/**</code> , <code>assets/images/**</code>	yes	large-file policy, manifest validation
<code>release-platform.yml</code>	tags like <code>release/*</code>	on tag	build and upload release assets
<code>sync-website-metadata.yml</code>	manual or on canon tag	optional	regenerate public-facing summaries for <code>dlrp.ca</code>

That structure uses GitHub Actions path filters correctly. GitHub's workflow syntax supports `paths` and `paths-ignore`, with the caveat that skipped required workflows can remain pending if branch protection expects them. Because of that, required checks should map to **always-created workflows with stable job names**, or you should use rulesets carefully so optional workflows do not block merges. GitHub's protected-branch guidance also warns that job names must be unique across workflows when they are used as required status checks. ²⁸

Concrete CI upgrades

The most useful near-term CI changes are these:

1. Add a **canon consistency job** that fails if the README version badge, README project card, and the current file under `canon/machine/` disagree on version or path. That would have caught the current `README v32.0` versus `master-plan-v30.2.json` mismatch immediately. ² ³
2. Add an **import smoke-test job** narrower than the full suite, for example:
3. `python -c "import huey.api"`
4. `python -c "from huey import cli"`
5. `pytest -q tests/test_env_validation.py`

This should run even if the broader suite remains unstable, because the repo's recent remediation summary explicitly notes that failing collection can hide regressions. ¹³

- 1. Add a **secret scanner** to match the documented policy; the security summary specifically notes no committed secret-scanner configuration was detected. ¹³
- 2. Split docs from runtime in CodeQL and artifact scans only where that improves signal. The current CodeQL already ignores docs/assets for some events, which is sensible. ²⁴

Governance and repository operating rules

The repository should adopt governance that matches its new physical structure. GitHub already provides the primitives: pull request templates, issue templates/forms, CODEOWNERS, protected branches, and rulesets. ²⁹

Recommended repository roles

Team size is unspecified, so the model should work for a solo maintainer and scale if collaborators are added later.

Role	Solo-repo reality	Responsibility
Founder / maintainer	Dylan holds it	final merge authority, public posture, release approval
Canon steward	same person or delegated later	law, glossary, master plan, README truth alignment
Runtime steward	same person or delegated later	<code>src/</code> , CLI/API, tests, import stability
Archive steward	same person or delegated later	historical payloads, lineage tags, manifests
Release steward	same person or delegated later	release assets, checksums, changelog
Security steward	same person or delegated later	secret handling, CI security, incident notes

PR templates

Create one default PR template and one migration-specific template.

Recommended default sections:

```
## Layer(s) touched
- canon
```



```

- implementation
- experiments
- history
- docs
- third_party
- ci

## What changed

## Canon impact
- [ ] none
- [ ] README updated
- [ ] master plan updated
- [ ] constitutional text updated

## Lineage impact
- [ ] no path moves
- [ ] git mv only
- [ ] compatibility pointer added
- [ ] release/reference links updated

## Artifact impact
- [ ] no binaries
- [ ] binaries added
- [ ] binaries moved to release assets
- [ ] manifest/checksum updated

## Verification
- tests:
- smoke checks:
- docs/canon checks:

```

GitHub supports repository pull request templates and issue templates on the default branch, and uses CODEOWNERS plus protected branches to standardize reviews. ³⁰

Commit message convention

A lightweight scoped convention is enough:

Prefix	Meaning
<code>canon:</code>	changes authoritative truth
<code>impl:</code>	runtime or infra behavior change
<code>docs:</code>	explanatory docs only
<code>exp:</code>	experiment/prototype only

Prefix	Meaning
<code>archive:</code>	historical move or archival action
<code>deps:</code>	dependency or third-party update
<code>move:</code>	structure-only path move with no content change
<code>ci:</code>	workflow/ruleset/check change
<code>release:</code>	tag/release manifest/assets

For structural moves, prefer messages that explicitly say **no content change** when true, for example:

```
move(canon): relocate law texts without content changes .
```

Archival policy

Use a simple content-state rule:

State	Rule
Active canon	editable, reviewed, versioned
Active implementation	editable, tested, reviewed
Experimental	editable, but never cited as source of truth
Historical	frozen by default; edits limited to pointer notes, metadata, and redaction if required
Release assets	immutable once published except for clear supersession notes

Tagging policy should reflect the layers:

Tag family	Use
<code>canon/v32.0</code>	authoritative canon snapshot
<code>runtime/v1.0.0-alpha.1</code>	runnable implementation snapshot
<code>release/hueyos-6.18.5</code>	public release payload tag
<code>archive/pre-layer-split-2026-05-03</code>	structural rollback point

GitHub Releases are built on tags, so this tagging split maps cleanly to GitHub's native release model. 31

Risks, rollback, and limitations

The main reorganization risks are predictable.

Risk	Why it matters	Mitigation
Broken imports	current runtime/tests already use mixed namespaces and shims	do structural split before namespace cleanup
Broken external links	website, README, docs, and scripts may reference old paths	maintain mapping table, pointer READMEs, release notes
Submodule breakage	moving submodules changes <code>.gitmodules</code> expectations	use <code>git mv</code> , then <code>git submodule sync && git submodule update --init --recursive</code>
CI deadlocks	path-filtered workflows can leave required checks pending	use stable always-present required jobs or aligned rulesets
Artifact loss or confusion	release payloads are part of lineage	baseline tag, manifest files, GitHub Releases, checksums
Canon drift surviving the move	moving files alone will not reconcile truth	add canon-consistency CI and designate a canon steward

Rollback plan

1. Before any move, create and push a signed or at least immutable baseline tag such as `pre-layer-split-2026-05-03`.
2. Land the reorg via a **single merge commit** from `reorg/layers-2026-05` into `main`, even if the reorg branch itself used many commits.
3. If rollback is needed after merge, revert the merge commit:

```
git checkout main
git revert -m 1 <merge-commit-sha>
git push origin main
```

1. If the reorg branch itself becomes confused before merge, reset it hard to the baseline tag and restart selectively:

```
git checkout reorg/layers-2026-05
git reset --hard pre-layer-split-2026-05-03
```

1. Keep a checked-in `docs/contributor/path-map.md` so even a reverted branch preserves the understood lineage mapping.

Open questions and limitations

A few details remain explicitly **unspecified** in the accessible evidence and should be treated as options, not facts:

- **Exact full per-file appendix:** I verified the repo's structure, major families, and many exact files, but I could not retrieve a fully lossless live recursive tree with exact size and last-modified metadata for every path through the exposed connector interfaces. The repo's own generated inventory and artifact report make the high-level structure clear, but a local scripted export is still the right way to produce the final CSV-grade appendix. ¹⁵ ⁴
- **Preferred Python import namespace:** `huey` versus `hueyos` should be resolved in phase two, after structure cleanup, because the current tree shows both names in live use. ¹² ¹⁰ ¹¹
- **Whether offline installer pools are first-class product goals:** if yes, some `.udeb` /boot payloads may remain in Git or LFS; if not, they should move to Releases/object storage. ⁴
- **LFS entitlement and bandwidth plan:** GitHub LFS limits depend on account plan; if that is not yet decided, prefer Releases for larger public payloads and reserve LFS for assets that genuinely need checkout-time fetch semantics. ³²

Prioritized references

Priority	Source	Why it matters
highest	live repository files via GitHub connector	direct evidence of current README, master plan, CI, tests, CODEOWNERS, submodule config, and artifact policy
highest	<code>dlrp.ca</code> project, software, and download pages	public-facing priority source and narrative baseline
high	GitHub Docs on large files, Releases, LFS, CODEOWNERS, branch protection, rulesets, templates, workflow syntax	primary operational guidance for the exact migration/repo-governance questions
high	<code>git-scm.com</code> docs for <code>git mv</code> , <code>git submodule</code> , <code>gitmodules</code>	primary mechanics for history-preserving moves and submodule relocation
medium	repo-internal restructure docs and artifact report	confirms existing intent and identifies already-known pressure points

Key sources used in this report include the live `README.md`, `master-plan-v30.2.json`, `ARTIFACT_SURFACE.md`, `pyproject.toml`, test files, workflow files, `.gitattributes`, `.gitmodules`, CODEOWNERS, the repo's own restructuring notes, and the current `dlrp.ca` project/download/software pages, along with official GitHub and Git documentation on repository limits, LFS, Releases, CODEOWNERS, protected branches, rulesets, templates, workflow syntax, `git mv`, and submodules. ² ³ ⁴ ¹² ¹⁰ ¹¹ ⁵ ¹⁷ ¹⁸ ⁷ ⁸ ³³

- 1 33 <https://dlrp.ca/projects/monkey-head-project-hueyos/>
<https://dlrp.ca/projects/monkey-head-project-hueyos/>
- 2 **README.md**
<https://github.com/DylanLRPollock/Monkey-Head-Project/blob/main/README.md>
- 3 **docs/unsorted/honeycomb-storage.md**
<https://github.com/DylanLRPollock/Monkey-Head-Project/blob/8e83e1ab972dceb3c04024c49b3540f3ce21aa3e/docs/unsorted/honeycomb-storage.md>
- 4 **ARTIFACT_SURFACE.md**
https://github.com/DylanLRPollock/Monkey-Head-Project/blob/main/ARTIFACT_SURFACE.md
- 5 **.gitattributes**
<https://github.com/DylanLRPollock/Monkey-Head-Project/blob/main/.gitattributes>
- 6 16 22 <https://docs.github.com/en/repositories/working-with-files/managing-large-files/about-large-files-on-github?platform=linux&ref=rahul-nath>
<https://docs.github.com/en/repositories/working-with-files/managing-large-files/about-large-files-on-github?platform=linux&ref=rahul-nath>
- 7 **repository-restructure-recommendation.md**
<https://github.com/DylanLRPollock/Monkey-Head-Project/blob/main/docs/unsorted/repository-restructure-recommendation.md>
- 8 **repository-restructure-inventory.md**
<https://github.com/DylanLRPollock/Monkey-Head-Project/blob/main/docs/unsorted/repository-restructure-inventory.md>
- 9 20 <https://git-scm.com/docs/git-mv.html>
<https://git-scm.com/docs/git-mv.html>
- 10 **test_cli.py**
https://github.com/DylanLRPollock/Monkey-Head-Project/blob/main/tests/test_cli.py
- 11 **test_api_routes.py**
https://github.com/DylanLRPollock/Monkey-Head-Project/blob/main/tests/test_api_routes.py
- 12 **pyproject.toml**
<https://github.com/DylanLRPollock/Monkey-Head-Project/blob/main/pyproject.toml>
- 13 **SECURITY_REMEDIATION_SUMMARY.md**
https://github.com/DylanLRPollock/Monkey-Head-Project/blob/main/SECURITY_REMEDIATION_SUMMARY.md
- 14 **api.py**
<https://github.com/DylanLRPollock/Monkey-Head-Project/blob/main/src/huey/memory/PY/api.py>
- 15 **git-ls-files.pass-01.txt**
<https://github.com/DylanLRPollock/Monkey-Head-Project/blob/main/.migration/inventory/git-ls-files.pass-01.txt>
- 17 **.gitmodules**
<https://github.com/DylanLRPollock/Monkey-Head-Project/blob/main/.gitmodules>
- 18 **CODEOWNERS**
<https://github.com/DylanLRPollock/Monkey-Head-Project/blob/main/.github/CODEOWNERS>
- 19 <https://dlrp.ca/download/>
<https://dlrp.ca/download/>

21 29 <https://docs.github.com/articles/about-code-owners>

<https://docs.github.com/articles/about-code-owners>

23 [ci.yml](#)

<https://github.com/DylanLRPollock/Monkey-Head-Project/blob/main/.github/workflows/ci.yml>

24 [codeql.yml](#)

<https://github.com/DylanLRPollock/Monkey-Head-Project/blob/main/.github/workflows/codeql.yml>

25 [security-bandit.yml](#)

<https://github.com/DylanLRPollock/Monkey-Head-Project/blob/main/.github/workflows/security-bandit.yml>

26 [security-dependency-scan.yml](#)

<https://github.com/DylanLRPollock/Monkey-Head-Project/blob/main/.github/workflows/security-dependency-scan.yml>

27 [security-artifact-container-scan.yml](#)

<https://github.com/DylanLRPollock/Monkey-Head-Project/blob/main/.github/workflows/security-artifact-container-scan.yml>

28 [https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax?](https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax?utm_cta=website-webinars-vhol-feature)

[utm_cta=website-webinars-vhol-feature](https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax?utm_cta=website-webinars-vhol-feature)

https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax?utm_cta=website-webinars-vhol-feature

30 <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/getting-started/managing-and-standardizing-pull-requests>

<https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/getting-started/managing-and-standardizing-pull-requests>

31 <https://docs.github.com/repositories/releasing-projects-on-github/about-releases>

<https://docs.github.com/repositories/releasing-projects-on-github/about-releases>

32 <https://docs.github.com/en/repositories/working-with-files/managing-large-files/about-git-large-file-storage?apiVersion=2022-11-28>

<https://docs.github.com/en/repositories/working-with-files/managing-large-files/about-git-large-file-storage?apiVersion=2022-11-28>